

Bioinformatics Protocol

Step 1: Downloading the human genome reference fasta file and the GTF annotation file

The genome FASTA and the GTF must use the same chromosome names and the same assembly.

Source of ref fasta and gtf files:

Release page: https://www.encodegenes.org/human/release_48.html

1. Genome FASTA

https://ftp.ebi.ac.uk/pub/databases/gencode/Gencode_human/release_48/GRCh38.primary_assembly.genome.fa.gz

Genome sequence, primary assembly (GRCh38)	PRI	- Nucleotide sequence of the GRCh38 primary genome assembly (chromosomes and scaffolds) - The sequence region names are the same as in the GTF/GFF3 files	Fasta
--	-----	---	-------

2. Annotation GTF

https://ftp.ebi.ac.uk/pub/databases/gencode/Gencode_human/release_48/gencode.v48.annotation.gtf.gz

GTF / GFF3 files

Content	Regions	Description	Download
Comprehensive gene annotation	CHR	It contains the comprehensive gene annotation on the reference chromosomes only	GTF GFF3

Step 2: Downloading the FastQ files

Link- <https://www.ebi.ac.uk/ena/browser/view/PRJNA647871>

Used the **wget** code for the download: `fastq_dnwd.sh`

Step 3: Downloading the metadata

Link- <https://www.ebi.ac.uk/ena/browser/view/PRJNA647871>

The metadata was downloaded from the .tsv file.

Read Files ?

Show Column Selection ▼

Download report: JSON TSV

Get download script

Download selected files

Step 4: Creating the conda environments

Environment list

The YAML files can be used to recreate the software environment.

1. **auris_core_v2.yml:** Read alignment, transcript assembly and gene quantification
2. **auris_lnc_v2.yml:** Novel-lncRNA identification, annotation and coding-potential filtering

Step 5: Check the fasta-gtf compatibility

Quick check

```
> grep -v '^#' gencode.v48.annotation.gtf | cut -f1 | sort -u > gtf_chr.txt
> grep '>' GRCh38.primary_assembly.genome.fa | sed 's/>//; s/ .*//' | sort -u > fa_chr.txt
> comm -23 gtf_chr.txt fa_chr.txt
```

#Should return nothing if every chr in the GTF exist in the FASTA. It will print lines if a GTF chromosome is missing from the FASTA.

The fasta file contains chr 1-22, chrX, chrY, chrM and GL, KI contigs whereas the gtf annotation file contains annotation for chr 1-22, chrX, chrY, chrM. For compatibility purposes we need to delete the GL and KI contigs.

Deleting process of GL, KI contigs:

1. Activate the conda environment-auris_core_v2

```
> conda activate auris_core_v2
```

2. Extract chromosome names from the GTF

```
> awk -F '\t' '!/^#/ && NF>=9 {print $1}' gencode.v48.annotation.gtf | sort -u >
gencode_v48_contigs.txt
```

3. Create the filtered FASTA

```
> seqkit grep -f gencode_v48_contigs.txt GRCh38.primary_assembly.genome.fa >
GRCh38.gencode_v48.chromosomes.fa
```

4. Verify the sequence names:

```
> seqkit seq -n -i GRCh38.gencode_v48.chromosomes.fa > filtered_fasta_contigs.txt
> cat filtered_fasta_contigs.txt
> wc -l filtered_fasta_contigs.txt
```

Expected: 25 sequences.

5. Confirm FASTA - GTF compatibility

```
> comm -3 <(sort filtered_fasta_contigs.txt) <(sort gencode_v48_contigs.txt)
```

Expected: no output.

Step 6: Create the FASTA index

```
> samtools faidx GRCh38.gencode_v48.chromosomes.fa
```

Confirm:

```
> ls -lh GRCh38.gencode_v48.chromosomes.fa.fai
```

#The .fai file allows SAMtools and other programs to retrieve specific genomic regions quickly

Build the STAR genome index

We did it through a SLURM job on HPC (High-Performance Computing).

```
> mkdir -p STAR_index
```

STAR Indexing script: `star_index.sh`

Then ran - `dos2unix star_index.sh` (converting file, `star_index.sh` to Unix format or else it show error).

Submit to HPC terminal `sbatch star_index.sh`

Expected files type:

- Genome
- SA
- SAindex
- chrLength.txt
- chrName.txt
- genomeParameters.txt

Step 7: Quality screening, FastQC and MultiQC on the raw fastq files

```
> conda activate auris_core_v2
```

Went to the folder where all the fastq files are and ran: `fastqc -t 8 -o . *.fastq.gz`

Then `multiqc . -o multiqc_raw`

Check the output from multiqc - `multiqc_report.html`

Step 8: Trimming by cutadapt

Cutadapt script: cutadapt_quantseq.sh

```
> sbatch cutadapt_quantseq.sh
```

Step 9: Again, did fastqc and multiqc

```
> mkdir -p /project/bishalab/msarker/c.auris_new/fastqc_trimmed  
> fastqc -t 8 -o /project/bishalab/msarker/c.auris_new/fastqc_trimmed  
/project/bishalab/msarker/c.auris_new/cutadapt/trimmed/*.trimmed.fastq.gz  
> multiqc /project/bishalab/msarker/c.auris_new/fastqc_trimmed -o  
/project/bishalab/msarker/c.auris_new/fastqc_trimmed -n multiqc_trimmed.html
```

The final report will be:

```
/project/bishalab/msarker/c.auris_new/fastqc_trimmed/multiqc_trimmed.html
```

Step 10: STAR alignment

```
> conda activate auris_core_v2  
> mkdir -p /project/bishalab/msarker/c.auris_new/STAR_alignment
```

Created the STAR_alignment.sh

```
> dos2unix STAR_alignment.sh  
> sbatch STAR_alignment.sh
```

Confirm 24 BAM files

```
find /project/bishalab/msarker/c.auris_new/STAR_alignment -name  
"*Aligned.sortedByCoord.out.bam" | wc -l
```

Expected: 24.

Check BAM integrity

```
samtools quickcheck -v  
/project/bishalab/msarker/c.auris_new/STAR_alignment/*Aligned.sortedByCoord.out.bam
```

Expected: no output.

Generate STAR MultiQC report

```
> mkdir -p /project/bishalab/msarker/c.auris_new/multiqc_star  
> multiqc /project/bishalab/msarker/c.auris_new/STAR_alignment -o  
/project/bishalab/msarker/c.auris_new/multiqc_star -n multiqc_star.html
```

Step 11: Confirm strand orientation

```
> awk 'NR>4 {u+=$2; f+=$3; r+=$4} END {print "Unstranded:",u,"Forward:",f,"Reverse:",r}'  
/project/bishalab/msarker/c.auris_new/STAR_alignment/SRR12291324_ReadsPerGene.out.tab
```

Step 12: index the BAM files

Made the file- `bam_index.sh`

```
> dos2unix bam_index.sh
```

```
> sbatch bam_index.sh
```

Confirm 24 indexes:

```
> find /project/bishalab/msarker/c.auris_new/STAR_alignment -name "*.bam.bai" | wc -l
```

Expected: 24.

Step 13: StringTie assembly

```
> mkdir -p /project/bishalab/msarker/c.auris_new/stringtie_assembly/{gtf,abundance,logs}
```

Created `stringtie_assembly.sh`

```
> dos2unix stringtie_assembly.sh
```

```
> sbatch stringtie_assembly.sh
```

After completion, verified 24 non-empty GTFs:

```
> find /project/bishalab/msarker/c.auris_new/stringtie_assembly/gtf -name "*.gtf" -type f -size  
+0c | wc -l
```

Expected: 24.

We did not add `-e`; StringTie must be allowed to assemble potentially novel transcripts.

Step 14: StringTie merge

```
> mkdir -p /project/bishalab/msarker/c.auris_new/stringtie_merge
```

```
> find /project/bishalab/msarker/c.auris_new/stringtie_assembly/gtf -maxdepth 1 -name "*.gtf" -  
type f | sort > /project/bishalab/msarker/c.auris_new/stringtie_merge/mergelist.txt
```

Confirm:

```
wc -l /project/bishalab/msarker/c.auris_new/stringtie_merge/mergelist.txt
```

Expected: 24.

```
> stringtie --merge -p 8 -G /project/bishalab/msarker/c.auris_new/genencode.v48.annotation.gtf -m 200 -o /project/bishalab/msarker/c.auris_new/stringtie_merge/stringtie_merged.gtf /project/bishalab/msarker/c.auris_new/stringtie_merge/mergelist.txt
```

It produced one main output:

```
> stringtie_merge/stringtie_merged.gtf
```

Step 15: GFFCompare

Ran GFFCompare against the same GENCODE v48 reference.

First confirmed the merged GTF contains only the 25 expected chromosomes:

```
> awk '!/^#/ {print $1}' /project/bishalab/msarker/c.auris_new/stringtie_merge/stringtie_merged.gtf | sort -u
```

Created the output directory: `mkdir -p /project/bishalab/msarker/c.auris_new/gffcompare`

Created `gffcompare.sh`

Submitted to slurm: `sbatch gffcompare.sh`

Expected file types:

- gffcmp.annotated.gtf
- gffcmp.stats
- gffcmp.tracking
- gffcmp.loci
- *.tmap
- *.refmap

Step 16: Count the class codes

The next step will be to examine `gffcmp.stats` and count/extract class codes from the `.tmap` file, especially primary candidates `u` and `x`.

extracted the class codes

```
> awk 'FNR>1 {print $3}' /project/bishalab/msarker/c.auris_new/stringtie_merge/gffcmp.stringtie_merged.gtf.tmap | sort | uniq -c | sort -nr > /project/bishalab/msarker/c.auris_new/gffcompare/class_code_counts.txt
```

Step 17: Generate the ID lists; (U & X)

Output dir- /project/bishalab/msarker/ c.auris_new/gffcompare

UX transcript id

```
> awk 'FNR>1 && ($3=="u" || $3=="x") {print $5}'  
/project/bishalab/msarker/c.auris_new/stringtie_merge/gffcmp.stringtie_merged.gtf.tmap | sort -u >  
/project/bishalab/msarker/c.auris_new/gffcompare/primary_ux_transcript_ids.txt
```

Extracted the primary u,x transcripts from the GFFCompare annotated GTF

1. Extract u+x

```
> gffread /project/bishalab/msarker/c.auris_new/gffcompare/gffcmp.annotated.gtf --ids  
/project/bishalab/msarker/c.auris_new/gffcompare/primary_ux_transcript_ids.txt -T -F -o  
/project/bishalab/msarker/c.auris_new/gffcompare/primary_ux_with_chrM.gtf
```

2. Remove mitochondrial candidates (ChrM)

```
> awk '$1!="chrM"'  
/project/bishalab/msarker/c.auris_new/gffcompare/primary_ux_with_chrM.gtf >  
/project/bishalab/msarker/c.auris_new/gffcompare/primary_ux_no_chrM.gtf
```

3. Number successfully extracted before removing chrM:

```
> awk '$3=="transcript" {n++} END {print n+0}'  
/project/bishalab/msarker/c.auris_new/gffcompare/primary_ux_with_chrM.gtf
```

4. Number remaining after removing chrM:

```
> awk '$3=="transcript" {n++} END {print n+0}'  
/project/bishalab/msarker/c.auris_new/gffcompare/primary_ux_no_chrM.gtf
```

Step 18: Now perform structural validation, confirm spliced length ≥ 200 nt, and separate multi-exon from single-exon transcripts.

```
> mkdir -p /project/bishalab/msarker/c.auris_new/lncrna_filtering/01_structure
```

Validated the GTF and enforced the length ≥ 200 nt

```
> gffread -E -l 200 /project/bishalab/msarker/c.auris_new/gffcompare/primary_ux_no_chrM.gtf -  
T -F -o  
/project/bishalab/msarker/c.auris_new/lncrna_filtering/01_structure/primary_ux_len200.gtf 2>  
/project/bishalab/msarker/c.auris_new/lncrna_filtering/01_structure/gffread_validation.log
```

Count retained transcripts:

```
> awk '$3=="transcript" {n++} END {print n+0}'  
/project/bishalab/msarker/c.auris_new/lncrna_filtering/01_structure/primary_ux_len200.gtf
```

Generated the structural table

```
> Gffread  
/project/bishalab/msarker/c.auris_new/lncrna_filtering/01_structure/primary_ux_len200.gtf --  
table @id,@geneid,@chr,@start,@end,@strand,@numexons,@covlen >  
/project/bishalab/msarker/c.auris_new/lncrna_filtering/01_structure/transcript_structure.tsv
```

Inspected it

```
> head  
/project/bishalab/msarker/c.auris_new/lncrna_filtering/01_structure/transcript_structure.tsv
```

Columns 7 and 8 are exon count and spliced length.

Step 19: Separate multi-exon and single-exon IDs

Multi-exon ids

```
> awk '$7>=2 && $8>=200 {print $1}'  
/project/bishalab/msarker/c.auris_new/lncrna_filtering/01_structure/transcript_structure.tsv | sort  
-u > /project/bishalab/msarker/c.auris_new/lncrna_filtering/01_structure/multiexon_ids.txt
```

Single exon ids

```
> awk '$7==1 && $8>=200 {print $1}'  
/project/bishalab/msarker/c.auris_new/lncrna_filtering/01_structure/transcript_structure.tsv | sort  
-u > /project/bishalab/msarker/c.auris_new/lncrna_filtering/01_structure/singleexon_ids.txt
```

Check the numbers

```
> wc -l /project/bishalab/msarker/c.auris_new/lncrna_filtering/01_structure/multiexon_ids.txt  
/project/bishalab/msarker/c.auris_new/lncrna_filtering/01_structure/singleexon_ids.txt
```

Extract the candidate FASTA

```
> gffread  
/project/bishalab/msarker/c.auris_new/lncrna_filtering/01_structure/primary_ux_len200.gtf -g  
/project/bishalab/msarker/c.auris_new/GRCh38.gencode_v48.chromosomes.fa -w  
/project/bishalab/msarker/c.auris_new/lncrna_filtering/01_structure/primary_ux_len200.fa
```

Checked

seqkit stats

/project/bishalab/msarker/c.auris_new/lncrna_filtering/01_structure/primary_ux_len200.fa

Step 20: Separate the candidates into multi-exon and single-exon branches.

DIR=/project/bishalab/msarker/c.auris_new/lncrna_filtering/01_structure

Extracted the multi-exon GTF

```
> gffread "$DIR/primary_ux_len200.gtf" --ids "$DIR/multiexon_ids.txt" -T -F -o  
"$DIR/primary_ux_multiexon.gtf"
```

Extracted the single-exon GTF

```
> gffread "$DIR/primary_ux_len200.gtf" --ids "$DIR/singleexon_ids.txt" -T -F -o  
"$DIR/primary_ux_singleexon.gtf"
```

Extracted their FASTA sequences

```
> gffread "$DIR/primary_ux_multiexon.gtf" -g  
/project/bishalab/msarker/c.auris_new/GRCh38.gencode_v48.chromosomes.fa -w  
"$DIR/primary_ux_multiexon.fa"
```

```
> gffread "$DIR/primary_ux_singleexon.gtf" -g  
/project/bishalab/msarker/c.auris_new/GRCh38.gencode_v48.chromosomes.fa -w  
"$DIR/primary_ux_singleexon.fa"
```

Checked the resulting numbers

```
> seqkit stats "$DIR/primary_ux_multiexon.fa" "$DIR/primary_ux_singleexon.fa"
```

Step 21: Junction support and poly(A)-priming validation

After this, the next major step is,

For the multi-exons,

1. validating the splice-junction support using the 24 STAR SJ>out.tab files.
2. internal-poly(A)-priming screening

For the single exons,

1. internal-poly(A)-priming screening

Before that, first confirming that all 24 STAR junction files exist:

```
find /project/bishalab/msarker/c.auris_new/STAR_alignment -maxdepth 1 -name "*_SJ.out.tab" |  
sort | wc -l
```

Expected output: 24

These **SJ.out.tab** files contain the observed splice junction coordinates and supporting read numbers.

Generated the required intron coordinates for the 196 multi-exon candidates:

```
> JDIR=/project/bishalab/msarker/c.auris_new/lncrna_filtering/02_junction_support
> mkdir -p "$JDIR"
> gffread
/project/bishalab/msarker/c.auris_new/lncrna_filtering/01_structure/primary_ux_multiexon.gtf --
table @id,@chr,@strand,@introns > "$JDIR/multiexon_candidate_introns.tsv"
```

Check it

```
wc -l "$JDIR/multiexon_candidate_introns.tsv"
```

Now compare the candidate introns against all 24 STAR junction files.

Defined the paths:

```
JDIR=/project/bishalab/msarker/c.auris_new/lncrna_filtering/02_junction_support
```

```
SJDIR=/project/bishalab/msarker/c.auris_new/STAR_alignment
```

Combine STAR junction support

```
awk 'BEGIN{OFS="\t"} {k=$1 OFS $2 OFS $3 OFS $4; uniq[k]+=$7; multi[k]+=$8; if($7>0)
samples[k]++; if($9>overhang[k]) overhang[k]=$9; motif[k]=$5} END{for(k in uniq) print
k,motif[k],samples[k],uniq[k],multi[k],overhang[k]}' "$SJDIR"/*_SJ.out.tab | sort -k1,1V -k2,2n
-k3,3n -k4,4n > "$JDIR/all_star_junction_support.tsv"
```

Expand candidate intron chains: This creates one row per transcript-intron combination

```
awk -F"\t" 'BEGIN{OFS="\t"} {gsub(/;/,"",$4); n=split($4,a,""); strand=($3=="+")?1:2;
for(i=1;i<=n;i++){split(a[i],b,"-"); print $1,$2,b[1],b[2],strand}}'
"$JDIR/multiexon_candidate_introns.tsv" > "$JDIR/candidate_introns_expanded.tsv"
```

Apply the junction-support criteria

An intron passes when it has:

- canonical motif code 1- 6
- uniquely mapped reads in at least 2 samples
- at least 3 uniquely mapped reads in total
- maximum alignment overhang ≥ 12 nt

```
> awk -F'\t' 'BEGIN{OFS="\t"; print
"transcript_id","chromosome","intron_start","intron_end","strand","motif","supporting_samples"
,"total_unique_reads","max_overhang","status"} NR==FNR{k=$1 SUBSEP $2 SUBSEP $3
SUBSEP $4; motif[k]=$5; samples[k]=$6; uniq[k]=$7; overhang[k]=$9; next} {k=$2 SUBSEP
$3 SUBSEP $4 SUBSEP $5; c=(k in motif)?motif[k]:0; n=(k in samples)?samples[k]:0; u=(k in
uniq)?uniq[k]:0; o=(k in overhang)?overhang[k]:0; status=(c>=1 && c<=6 && n>=2 && u>=3
&& o>=12)?"PASS":"FAIL"; strand=($5==1)?"+": "-"; print $1,$2,$3,$4,strand,c,n,u,o,status}'
"$JDIR/all_star_junction_support.tsv" "$JDIR/candidate_introns_expanded.tsv" >
"$JDIR/intron_support_report.tsv"
```

Retained transcripts only when every intron passed

```
> awk -F'\t' 'NR>1 {seen[$1]=1; if($10!="PASS") bad[$1]=1} END{for(t in seen) if(!(t in bad))
print t}' "$JDIR/intron_support_report.tsv" | sort -u >
"$JDIR/junction_supported_multiexon_ids.txt"
```

```
> awk -F'\t' 'NR>1 {seen[$1]=1; if($10!="PASS") bad[$1]=1} END{for(t in seen) if(t in bad)
print t}' "$JDIR/intron_support_report.tsv" | sort -u > "$JDIR/junction_failed_multiexon_ids.txt"
```

Check the result

```
> wc -l "$JDIR/junction_supported_multiexon_ids.txt"
"$JDIR/junction_failed_multiexon_ids.txt"
```

Then extracted only the junction-supported multi-exon transcripts. These codes store paths for later commands.

```
> SDIR=/project/bishalab/msarker/c.auris_new/lncrna_filtering/01_structure
> JDIR=/project/bishalab/msarker/c.auris_new/lncrna_filtering/02_junction_support
> GENOME=/project/bishalab/msarker/c.auris_new/GRCh38.gencode_v48.chromosomes.fa
```

Extracted the supported GTF

```
> gffread "$SDIR/primary_ux_multiexon.gtf" --ids
"$JDIR/junction_supported_multiexon_ids.txt" -T -F -o
"$JDIR/primary_ux_multiexon_junction_supported.gtf"
```

Extracted its transcript sequences

```
> gffread "$JDIR/primary_ux_multiexon_junction_supported.gtf" -g "$GENOME" -w
"$JDIR/primary_ux_multiexon_junction_supported.fa"
```

Step 22: Next is internal-poly(A)-priming screening

1. Define paths

```
> SDIR=/project/bishalab/msarker/c.auris_new/lncrna_filtering/01_structure
> JDIR=/project/bishalab/msarker/c.auris_new/lncrna_filtering/02_junction_support
> PDIR=/project/bishalab/msarker/c.auris_new/lncrna_filtering/03_internal_priming
> GENOME=/project/bishalab/msarker/c.auris_new/GRCh38.gencode_v48.chromosomes.fa
> mkdir -p "$PDIR"
```

2. Obtained the 20 genomic bases downstream of each 3' end

Supported multi-exon candidates:

```
> awk -F'\t' 'BEGIN{OFS="\t"} $3=="transcript"{split($9,a,"transcript_id \"); split(a[2],b,"\");
id=b[1]; if($7=="+") print $1,$5,$5+20,id,0,$7; else if($7=="-"){s=$4-21; if(s<0)s=0; print
$1,s,$4-1,id,0,$7}}' "$JDIR/primary_ux_multiexon_junction_supported.gtf" >
"$PDIR/multiexon_downstream20.bed"
```

Single-exon candidates:

```
> awk -F'\t' 'BEGIN{OFS="\t"} $3=="transcript"{split($9,a,"transcript_id \"); split(a[2],b,"\");
id=b[1]; if($7=="+") print $1,$5,$5+20,id,0,$7; else if($7=="-"){s=$4-21; if(s<0)s=0; print
$1,s,$4-1,id,0,$7}}' "$SDIR/primary_ux_singleexon.gtf" >
"$PDIR/singleexon_downstream20.bed"
```

3. Extract the strand-oriented sequences

```
> bedtools getfasta -fi "$GENOME" -bed "$PDIR/multiexon_downstream20.bed" -s -nameOnly
-fo "$PDIR/multiexon_downstream20.fa"

> bedtools getfasta -fi "$GENOME" -bed "$PDIR/singleexon_downstream20.bed" -s -nameOnly
-fo "$PDIR/singleexon_downstream20.fa"
```

4. Flagged A-rich regions

A candidate is flagged when the downstream 20 nt contain either:

- at least six consecutive A's;
- or
- at least 12 in total A's.

```
> for TYPE in multiexon singleexon; do awk 'BEGIN{OFS="\t"; print
"transcript_id","downstream_sequence","A_count","status"} /^>/{id=substr($0,2); next}
{seq=toupper($0); temp=seq; n=gsub(/A/,"",temp); status=(seq~/AAAAAA/ ||
n>=12)?"A_RICH":"PASS"; print id,seq,n,status}' "$PDIR/${TYPE}_downstream20.fa" >
"$PDIR/${TYPE}_internal_priming_report.tsv"; done
```

Summarize

```
> for TYPE in multiexon singleexon; do echo "$TYPE"; awk -F'\t' 'NR>1 {n[$4]++} END {for(x
in n) print x,n[x]}' "$PDIR/${TYPE}_internal_priming_report.tsv"; done
```

Separated the internal-priming PASS and A_RICH IDs.

```
> awk -F'\t' 'NR>1 && $4=="PASS" {print $1}' "$PDIR/multiexon_internal_priming_report.tsv" >
"$PDIR/multiexon_no_internal_priming_ids.txt"
```

```
> awk -F'\t' 'NR>1 && $4=="A_RICH" {print $1}'
"$PDIR/multiexon_internal_priming_report.tsv" > "$PDIR/multiexon_A_rich_ids.txt"
```

```
> awk -F'\t' 'NR>1 && $4=="PASS" {print $1}' "$PDIR/singleexon_internal_priming_report.tsv" >
"$PDIR/singleexon_no_internal_priming_ids.txt"
```

```
> awk -F'\t' 'NR>1 && $4=="A_RICH" {print $1}'
"$PDIR/singleexon_internal_priming_report.tsv" > "$PDIR/singleexon_A_rich_ids.txt"
```

Check the numbers

```
> wc -l "$PDIR"/*_no_internal_priming_ids.txt "$PDIR"/*_A_rich_ids.txt
```

Now we extracted the 676 non-A-rich candidates (22 from multi-exon and 654 from single exon)

Clean up needed: The IDs contain strand suffixes such as MSTRG.79.17(-) but gffread expects, MSTRG.79.17

Cleaned all four ID lists:

```
> for f in multiexon_no_internal_priming_ids singleexon_no_internal_priming_ids multiexon_A_rich_ids
singleexon_A_rich_ids; do sed -E 's/\([+-]\)$//' "$PDIR/${f}.txt" | sort -u > "$PDIR/${f}.clean.txt"; done
```

Checked the cleaned counts

```
> wc -l "$PDIR"/*.clean.txt
```

Now extract the gtf files from the id list for multiexon and singleexon

```
> gffread "$JDIR/primary_ux_multiexon_junction_supported.gtf" --ids
"$PDIR/multiexon_no_internal_priming_ids.clean.txt" -T -F -o
"$PDIR/multiexon_no_internal_priming.gtf"
```

```
> gffread "$SDIR/primary_ux_singleexon.gtf" --ids
"$PDIR/singleexon_no_internal_priming_ids.clean.txt" -T -F -o
"$PDIR/singleexon_no_internal_priming.gtf"
```

Verify

```
> awk '$3=="transcript"{n++} END{print n+0}' "$PDIR/multiexon_no_internal_priming.gtf"
> awk '$3=="transcript"{n++} END{print n+0}' "$PDIR/singleexon_no_internal_priming.gtf"
```

Then combined the two retained branches into one provisional candidate GTF

```
> gffread "$PDIR/multiexon_no_internal_priming.gtf"
"$PDIR/singleexon_no_internal_priming.gtf" -T -F -o "$PDIR/primary_ux_structural_pass.gtf"
```

Confirm the total

```
> awk '$3=="transcript"{n++} END{print n+0}' "$PDIR/primary_ux_structural_pass.gtf"
```

Extracted their transcript sequences

```
> gffread "$PDIR/primary_ux_structural_pass.gtf" -g "$GENOME" -w
"$PDIR/primary_ux_structural_pass.fa"
```

Verify

```
> seqkit stats "$PDIR/primary_ux_structural_pass.fa"
```

These 676 consist of 22 supported multi-exon and 654 provisional single-exon candidates. They are not confirmed lncRNAs yet. The next stage is removing candidates representing tRNA, rRNA, miRNA, snoRNA, snRNA and other structured ncRNAs.

Step 23: Rfam structured-ncRNA screening.

```
> conda activate auris_lnc_v2
> cmscan -h >/dev/null && echo "Infernal is ready"
```

Download and prepare Rfam

```
> mkdir -p
/project/bishalab/msarker/c.auris_new/lncrna_filtering/04_rfam/{database,results,logs}
> wget -c https://ftp.ebi.ac.uk/pub/databases/Rfam/CURRENT/Rfam.cm.gz -O
/project/bishalab/msarker/c.auris_new/lncrna_filtering/04_rfam/database/Rfam.cm.gz
> wget -c https://ftp.ebi.ac.uk/pub/databases/Rfam/CURRENT/Rfam.clanin -O
/project/bishalab/msarker/c.auris_new/lncrna_filtering/04_rfam/database/Rfam.clanin
```

```
> gzip -dc /project/bishalab/msarker/c.auris_new/lncrna_filtering/04_rfam/database/Rfam.cm.gz >
/project/bishalab/msarker/c.auris_new/lncrna_filtering/04_rfam/database/Rfam.cm
```

Index the database once

```
> cmpress /project/bishalab/msarker/c.auris_new/lncrna_filtering/04_rfam/database/Rfam.cm
```

Create rfam_scan.sh

```
> dos2unix rfam_scan.sh
```

```
> sbatch rfam_scan.sh
```

Check after completion

```
> grep -v '^#' /project/bishalab/msarker/c.auris_new/lncrna_filtering/04_rfam/results/rfam.tblout |
sed '/^[[[:space:]]*$]/d' | wc -l
```

Count how many unique candidate transcripts have hits:

```
> awk '!/^#/ && NF {print $4}'
/project/bishalab/msarker/c.auris_new/lncrna_filtering/04_rfam/results/rfam.tblout | sort -u | wc -
l
```

Summarize the detected Rfam families:

```
> awk '!/^#/ && NF {print $2}'
/project/bishalab/msarker/c.auris_new/lncrna_filtering/04_rfam/results/rfam.tblout | sort | uniq -c
| sort -nr
```

Created the Rfam exclusion list

```
> awk '!/^#/ && NF {print $4}'
/project/bishalab/msarker/c.auris_new/lncrna_filtering/04_rfam/results/rfam.tblout | sed -E
's/^\([+-]\)$/' | sort -u >
/project/bishalab/msarker/c.auris_new/lncrna_filtering/04_rfam/results/rfam_exclude_ids.txt
```

Verify

```
> wc -l
/project/bishalab/msarker/c.auris_new/lncrna_filtering/04_rfam/results/rfam_exclude_ids.txt
```

Saved the excluded candidates for auditing

```
> gffread
/project/bishalab/msarker/c.auris_new/lncrna_filtering/03_internal_priming/primary_ux_structur
al_pass.gtf --ids
/project/bishalab/msarker/c.auris_new/lncrna_filtering/04_rfam/results/rfam_exclude_ids.txt -T -
F -o /project/bishalab/msarker/c.auris_new/lncrna_filtering/04_rfam/results/rfam_excluded.gtf
```

Removed them from the candidate GTF

```
> gffread
/project/bishalab/msarker/c.auris_new/lncrna_filtering/03_internal_priming/primary_ux_structural_pass.gtf --nids
/project/bishalab/msarker/c.auris_new/lncrna_filtering/04_rfam/results/rfam_exclude_ids.txt -T -F -o
/project/bishalab/msarker/c.auris_new/lncrna_filtering/04_rfam/results/primary_ux_rfam_pass.gtf
```

Expected remaining candidates

```
> awk '$3=="transcript" {n++} END {print n+0}'
/project/bishalab/msarker/c.auris_new/lncrna_filtering/04_rfam/results/primary_ux_rfam_pass.gtf
```

Extract the Rfam-passed FASTA

```
> gffread
/project/bishalab/msarker/c.auris_new/lncrna_filtering/04_rfam/results/primary_ux_rfam_pass.gtf -g /project/bishalab/msarker/c.auris_new/GRCh38.gencode_v48.chromosomes.fa -w
/project/bishalab/msarker/c.auris_new/lncrna_filtering/04_rfam/results/primary_ux_rfam_pass.fa

> seqkit stats
/project/bishalab/msarker/c.auris_new/lncrna_filtering/04_rfam/results/primary_ux_rfam_pass.fa
```

Step 24: Next essential step is CPC2 coding-potential screening.

Ran CPC2 on the 661 Rfam-passed transcripts.

verify CPC2:

```
> conda activate auris_lnc_v2
> CPC2.py -h >/dev/null && echo "CPC2 is ready"
```

It will print- “CPC2 is ready” if CPC2 is working fine.

Created directories:

```
> mkdir -p /project/bishalab/msarker/c.auris_new/lncrna_filtering/05_cpc2/{results,logs}
```

Saved this as `cpc2_scan.sh`:

Submit:

```
> dos2unix cpc2_scan.sh
```

```
> sbatch cpc2_scan.sh
```

The final CPC2 output will be:

```
> /project/bishalab/msarker/c.auris_new/lncrna_filtering/05_cpc2/results/cpc2_results.txt
```

CPC2 automatically adds .txt to the output name.

Summarize the labels

```
> awk -F'\t' 'NR>1 {n[tolower($NF)]++} END{for(x in n) print x,n[x]}'  
/project/bishalab/msarker/c.auris_new/lncrna_filtering/05_cpc2/results/cpc2_results.txt
```

Created separate ID lists:

```
> awk -F'\t' 'NR>1 && tolower($NF)=="noncoding">{print $1}'  
/project/bishalab/msarker/c.auris_new/lncrna_filtering/05_cpc2/results/cpc2_results.txt | sort -u >  
/project/bishalab/msarker/c.auris_new/lncrna_filtering/05_cpc2/results/cpc2_noncoding_ids.txt  
  
> awk -F'\t' 'NR>1 && tolower($NF)=="coding">{print $1}'  
/project/bishalab/msarker/c.auris_new/lncrna_filtering/05_cpc2/results/cpc2_results.txt | sort -u >  
/project/bishalab/msarker/c.auris_new/lncrna_filtering/05_cpc2/results/cpc2_coding_ids.txt
```

Verify

```
> wc -l /project/bishalab/msarker/c.auris_new/lncrna_filtering/05_cpc2/results/cpc2_*_ids.txt
```

Extracted the 659 noncoding candidates

```
> gffread  
/project/bishalab/msarker/c.auris_new/lncrna_filtering/04_rfam/results/primary_ux_rfam_pass.gtf  
f --ids  
/project/bishalab/msarker/c.auris_new/lncrna_filtering/05_cpc2/results/cpc2_noncoding_ids.txt -  
T -F -o  
/project/bishalab/msarker/c.auris_new/lncrna_filtering/05_cpc2/results/primary_ux_cpc2_nonco  
ding.gtf
```

Saved the two excluded coding candidates separately

```
> gffread  
/project/bishalab/msarker/c.auris_new/lncrna_filtering/04_rfam/results/primary_ux_rfam_pass.gtf  
f --ids  
/project/bishalab/msarker/c.auris_new/lncrna_filtering/05_cpc2/results/cpc2_coding_ids.txt -T -F  
-o  
/project/bishalab/msarker/c.auris_new/lncrna_filtering/05_cpc2/results/cpc2_excluded_coding.gtf
```

Verify the retained count

```
> awk '$3=="transcript" {n++} END {print n+0}'  
/project/bishalab/msarker/c.auris_new/lncrna_filtering/05_cpc2/results/primary_ux_cpc2_noncoding.gtf
```

Expected: 659

Extract their sequences

```
> gffread  
/project/bishalab/msarker/c.auris_new/lncrna_filtering/05_cpc2/results/primary_ux_cpc2_noncoding.gtf -g /project/bishalab/msarker/c.auris_new/GRCh38.gencode_v48.chromosomes.fa -w /project/bishalab/msarker/c.auris_new/lncrna_filtering/05_cpc2/results/primary_ux_cpc2_noncoding.fa
```

Verify

```
> seqkit stats  
/project/bishalab/msarker/c.auris_new/lncrna_filtering/05_cpc2/results/primary_ux_cpc2_noncoding.fa
```

Expected-659

Step 25: Pfam scan

Pfam searches for amino-acid sequences rather than nucleotide sequences. Therefore, ORFs were predicted from the 659 CPC2-classified noncoding transcripts using orfipy. Because the transcript FASTA sequences were already oriented in the direction of transcription, ORFs were searched only on the forward strand. ORFs were required to begin with ATG and have a minimum length of 30 nucleotides. Partial ORFs lacking a terminal stop codon were permitted because the QuantSeq libraries were enriched for transcript 3' ends.

```
> conda activate auris_lnc_v2  
> conda install -n auris_lnc_v2 -c conda-forge -c bioconda --strict-channel-priority orfipy -y
```

Confirm Conda installed it:

```
> conda list orfipy  
> command -v orfipy
```

generate forward-strand ORF protein sequences for Pfam:

```
> mkdir -p  
/project/bishalab/msarker/c.auris_new/lncrna_filtering/06_pfam/{database,results,logs}
```

```
> orfipy  
/project/bishalab/msarker/c.auris_new/lncrna_filtering/05_cpc2/results/primary_ux_cpc2_noncoding.fa --pep primary_ux_candidate_orfs.pep.fa --min 30 --strand f --start ATG --partial-3 --table 1 --procs 8 --outdir /project/bishalab/msarker/c.auris_new/lncrna_filtering/06_pfam/results
```

Verify

```
> seqkit stats  
/project/bishalab/msarker/c.auris_new/lncrna_filtering/06_pfam/results/primary_ux_candidate_orfs.pep.fa
```

For each transcript containing at least one qualifying ORF, the longest predicted ORF was retained and translated for Pfam analysis. Transcripts without a qualifying ORF remained in the noncoding candidate set.

Created the longest-ORF ID list

```
> seqkit fx2tab -i -l  
/project/bishalab/msarker/c.auris_new/lncrna_filtering/06_pfam/results/primary_ux_candidate_orfs.pep.fa | awk -F\t '{parent=$1; sub(/_ORF\.[0-9]+$/, "", parent);  
if($2>max[parent]){max[parent]=$2; best[parent]=$1}} END{for(parent in best) print  
best[parent]}' | sort >  
/project/bishalab/msarker/c.auris_new/lncrna_filtering/06_pfam/results/longest_orf_ids.txt
```

Extracted those proteins

```
> seqkit grep -f  
/project/bishalab/msarker/c.auris_new/lncrna_filtering/06_pfam/results/longest_orf_ids.txt  
/project/bishalab/msarker/c.auris_new/lncrna_filtering/06_pfam/results/primary_ux_candidate_orfs.pep.fa -o  
/project/bishalab/msarker/c.auris_new/lncrna_filtering/06_pfam/results/primary_ux_longest_orfs.pep.fa
```

verify

```
seqkit stats  
/project/bishalab/msarker/c.auris_new/lncrna_filtering/06_pfam/results/primary_ux_longest_orfs.pep.fa
```

For example, if one transcript has 14-aa, 17-aa and 30-aa ORFs, only its 30-aa ORF is retained.

Next ran the Pfam protein-domain screen.

Activated the correct environment and verify HMMER:

```
> conda activate auris_lnc_v2
```

```
> hmmscan -h >/dev/null && echo "HMMER is ready"
```

Download the current Pfam-A database (release 38.2):

```
wget -c https://ftp.ebi.ac.uk/pub/databases/Pfam/current\_release/Pfam-A.hmm.gz -O  
/project/bishalab/msarker/c.auris_new/lncrna_filtering/06_pfam/database/Pfam-A.hmm.gz
```

Create `pfam_scan.sh` with:

Submit it:

```
dos2unix pfam_scan.sh
```

```
sbatch pfam_scan.sh
```

Then counted significant Pfam domain hits:

```
> awk '!/^#/ && NF {n++} END{print n+0}'  
/project/bishalab/msarker/c.auris_new/lncrna_filtering/06_pfam/results/pfam.domtblout
```

Count unique ORFs with hits:

```
> awk '!/^#/ && NF {print $4}'  
/project/bishalab/msarker/c.auris_new/lncrna_filtering/06_pfam/results/pfam.domtblout | sort -u |  
wc -l
```

Create and count unique transcript IDs with Pfam hits:

```
> awk '!/^#/ && NF {id=$4; sub(/_ORF\[0-9\]+$/, "", id); print id}'  
/project/bishalab/msarker/c.auris_new/lncrna_filtering/06_pfam/results/pfam.domtblout | sort -u >  
/project/bishalab/msarker/c.auris_new/lncrna_filtering/06_pfam/results/pfam_hit_transcript_ids.txt
```

Inspected the hit:

```
> grep -v '^#'  
/project/bishalab/msarker/c.auris_new/lncrna_filtering/06_pfam/results/pfam.domtblout | sed  
'/^[[[:space:]]*$/d'
```

Confirm its transcript ID:

```
> cat  
/project/bishalab/msarker/c.auris_new/lncrna_filtering/06_pfam/results/pfam_hit_transcript_ids.t  
xt
```

Created the Pfam-pass ID list:

```
> comm -23 <(sort  
/project/bishalab/msarker/c.auris_new/lncrna_filtering/05_cpc2/results/cpc2_noncoding_ids.txt)  
<(sort
```

```
/project/bishalab/msarker/c.auris_new/lncrna_filtering/06_pfam/results/pfam_hit_transcript_ids.txt) > /project/bishalab/msarker/c.auris_new/lncrna_filtering/06_pfam/results/pfam_pass_ids.txt
```

Verify

```
> wc -l  
/project/bishalab/msarker/c.auris_new/lncrna_filtering/06_pfam/results/pfam_pass_ids.txt
```

Extract the retained GTF:

```
> gffread  
/project/bishalab/msarker/c.auris_new/lncrna_filtering/05_cpc2/results/primary_ux_cpc2_noncoding.gtf --ids  
/project/bishalab/msarker/c.auris_new/lncrna_filtering/06_pfam/results/pfam_pass_ids.txt -T -F -o  
/project/bishalab/msarker/c.auris_new/lncrna_filtering/06_pfam/results/primary_ux_pfam_pass.gtf
```

Saved the excluded transcript separately

```
> gffread  
/project/bishalab/msarker/c.auris_new/lncrna_filtering/05_cpc2/results/primary_ux_cpc2_noncoding.gtf --ids  
/project/bishalab/msarker/c.auris_new/lncrna_filtering/06_pfam/results/pfam_hit_transcript_ids.txt -T -F -o  
/project/bishalab/msarker/c.auris_new/lncrna_filtering/06_pfam/results/pfam_excluded.gtf
```

Extract the retained sequences

```
> gffread  
/project/bishalab/msarker/c.auris_new/lncrna_filtering/06_pfam/results/primary_ux_pfam_pass.gtf -g /project/bishalab/msarker/c.auris_new/GRCh38.gencode_v48.chromosomes.fa -w  
/project/bishalab/msarker/c.auris_new/lncrna_filtering/06_pfam/results/primary_ux_pfam_pass.fasta
```

Verified, both reports 658

```
> awk '$3=="transcript"{n++} END{print n+0}'  
/project/bishalab/msarker/c.auris_new/lncrna_filtering/06_pfam/results/primary_ux_pfam_pass.gtf  
  
> seqkit stats  
/project/bishalab/msarker/c.auris_new/lncrna_filtering/06_pfam/results/primary_ux_pfam_pass.fasta
```

658 candidates passed Rfam, CPC2 and Pfam

Step 26: strand-aware coding-gene 3'-fragment screen

The next essential filter is the strand-aware coding-gene 3'-fragment screen. Because this is QuantSeq 3' RNA-seq, a candidate immediately downstream of a protein-coding gene on the same strand could be an unannotated extension rather than an independent lncRNA. The 2 kb is a practical high-risk boundary, not a universal biological cutoff.

Create the directory

```
> mkdir -p /project/bishalab/msarker/c.auris_new/lncrna_filtering/07_genomic_context
```

Create the candidate BED:

```
> awk -F'\t' 'BEGIN{OFS="\t"} $3=="transcript"{id=$9; sub(/.*transcript_id "/, "", id);  
sub(/".*/, "", id); print $1,$4-1,$5,id,".", $7}'  
/project/bishalab/msarker/c.auris_new/lncrna_filtering/06_pfam/results/primary_ux_pfam_pass.g  
tf >  
/project/bishalab/msarker/c.auris_new/lncrna_filtering/07_genomic_context/pfam_pass_candidat  
es.bed
```

Create a protein-coding gene BED

```
> awk -F'\t' 'BEGIN{OFS="\t"} $3=="gene" && $0~/gene_type "protein_coding"/{id=$9;  
sub(/.*gene_id "/, "", id); sub(/".*/, "", id); print $1,$4-1,$5,id,".", $7}'  
/project/bishalab/msarker/c.auris_new/genencode.v48.annotation.gtf >  
/project/bishalab/msarker/c.auris_new/lncrna_filtering/07_genomic_context/protein_coding_gen  
es.bed
```

Create strand-aware 2-kb regions downstream of coding genes:

```
> awk 'BEGIN{OFS="\t"} $6=="+"{print $1,$3,$3+2000,$4,".", $6} $6=="-" {s=$2-2000;  
if(s<0)s=0; print $1,s,$2,$4,".", $6}'  
/project/bishalab/msarker/c.auris_new/lncrna_filtering/07_genomic_context/protein_coding_gen  
es.bed >  
/project/bishalab/msarker/c.auris_new/lncrna_filtering/07_genomic_context/coding_gene_3prim  
e_2kb_windows.bed
```

Identify candidates overlapping these regions on the same strand:

```
> bedtools intersect -s -wa -wb -a  
/project/bishalab/msarker/c.auris_new/lncrna_filtering/07_genomic_context/pfam_pass_candidat  
es.bed -b  
/project/bishalab/msarker/c.auris_new/lncrna_filtering/07_genomic_context/coding_gene_3prim  
e_2kb_windows.bed >
```

```
/project/bishalab/msarker/c.auris_new/lncrna_filtering/07_genomic_context/coding_3prime_risk_pairs.tsv
```

Extract and count flagged candidates:

```
> cut -f4  
/project/bishalab/msarker/c.auris_new/lncrna_filtering/07_genomic_context/coding_3prime_risk_pairs.tsv | sort -u >  
/project/bishalab/msarker/c.auris_new/lncrna_filtering/07_genomic_context/coding_3prime_risk_ids.txt
```

```
> wc -l  
/project/bishalab/msarker/c.auris_new/lncrna_filtering/07_genomic_context/coding_3prime_risk_ids.txt
```

34 candidates are high-risk for being coding-gene 3' extensions. That is about 5.2% of the 658 candidates.

We did not delete them solely because of proximity. Separated them into:

- 624 low-risk candidates
- 34 candidates requiring later expression/IGV review

Create the low-risk ID list:

```
> comm -23  
/project/bishalab/msarker/c.auris_new/lncrna_filtering/06_pfam/results/pfam_pass_ids.txt  
/project/bishalab/msarker/c.auris_new/lncrna_filtering/07_genomic_context/coding_3prime_risk_ids.txt >  
/project/bishalab/msarker/c.auris_new/lncrna_filtering/07_genomic_context/coding_3prime_low_risk_ids.txt
```

Verify the split:

```
> wc -l  
/project/bishalab/msarker/c.auris_new/lncrna_filtering/07_genomic_context/coding_3prime_low_risk_ids.txt  
/project/bishalab/msarker/c.auris_new/lncrna_filtering/07_genomic_context/coding_3prime_risk_ids.txt
```

Extract the low-risk GTF:

```
> gffread  
/project/bishalab/msarker/c.auris_new/lncrna_filtering/06_pfam/results/primary_ux_pfam_pass.gtf --ids  
/project/bishalab/msarker/c.auris_new/lncrna_filtering/07_genomic_context/coding_3prime_low
```

```
risk_ids.txt -T -F -o
/project/bishalab/msarker/c.auris_new/lncrna_filtering/07_genomic_context/primary_ux_genomi
c_lowrisk.gtf
```

Save the 34 flagged candidates separately:

```
> gffread
/project/bishalab/msarker/c.auris_new/lncrna_filtering/06_pfam/results/primary_ux_pfam_pass.g
tf --ids
/project/bishalab/msarker/c.auris_new/lncrna_filtering/07_genomic_context/coding_3prime_risk
_ids.txt -T -F -o
/project/bishalab/msarker/c.auris_new/lncrna_filtering/07_genomic_context/primary_ux_3prime
_risk.gtf
```

For the upcoming expression screen, retain all 658 temporarily. Expression patterns and IGV coverage will determine whether any of the 34 are independent transcripts or coding-gene fragments.

Step 27: Expression-screen

Next, counted expression for all 658 candidates while keeping the 34 proximity-risk transcripts flagged for later IGV review.

Create the expression-screen directory:

```
> mkdir -p
/project/bishalab/msarker/c.auris_new/lncrna_filtering/08_expression_screen/{results,logs}
```

First determine how many unique novel loci the 658 transcripts represent:

```
> awk -F'\t' '$3=="transcript"{print $9}'
/project/bishalab/msarker/c.auris_new/lncrna_filtering/06_pfam/results/primary_ux_pfam_pass.g
tf | sed -n 's/.*gene_id "([^\"]*)".*/\1/p' | sort -u >
/project/bishalab/msarker/c.auris_new/lncrna_filtering/08_expression_screen/results/candidate_g
ene_ids.txt

> wc -l
/project/bishalab/msarker/c.auris_new/lncrna_filtering/08_expression_screen/results/candidate_g
ene_ids.txt
```

Output-657

Created the provisional counting annotation containing **full GENCODE** plus all **658 candidates**:

```
> cat /project/bishalab/msarker/c.auris_new/gencode.v48.annotation.gtf
/project/bishalab/msarker/c.auris_new/lncrna_filtering/06_pfam/results/primary_ux_pfam_pass.g
tf >
```

/project/bishalab/msarker/c.auris_new/lncrna_filtering/08_expression_screen/provisional_annotation_658.gtf

Create `screening_featurecounts.sh`

Submit:

`dos2unix screening_featurecounts.sh`

`sbatch screening_featurecounts.sh`

`screening_gene_counts.txt` was generated using: Full GENCODE annotation + 658 novel candidate transcripts.

Here, `-s 1` applies FWD strand orientation, and `-g gene_id` counts each MSTRG locus once. After completion, we will check assignment rates and perform the donor-reproducible expression filter in RStudio.

Output

`screening_gene_counts.txt`

`screening_gene_counts.txt.summary`

Step 28: Expression-based validation of the 658 candidates in R

R files (see R files listed below)

- `expression_screen.R`
- `expression_screen.RData`

Parameters: `cpm_threshold`: ≥ 0.5 & `minimum_donors`: 2

Results of the expression screening-based analysis

- Candidate IDs supplied: 657
- Candidate loci found: 657
- Candidate IDs missing: 0
- Novel candidate loci on chrM: 0
- Expression-pass loci: **613**
- Expression-failed loci: **44**

Step 29: getting the transcript ids gtf file for the 613 expression-passing loci

Converted the **windows formatted R** file into **unix** format

`> dos2unix`

/project/bishalab/msarker/c.auris_new/lncrna_filtering/08_expression_screen/results/expression_pass_gene_ids.txt

Convert the 613 gene IDs to transcript IDs

```
> awk 'NR==FNR {keep[$1]=1; next} ($2 in keep){print $1}'  
/project/bishalab/msarker/c.auris_new/lncrna_filtering/08_expression_screen/results/expression_  
pass_gene_ids.txt  
/project/bishalab/msarker/c.auris_new/lncrna_filtering/06_pfam/results/pfam_pass_transcript_ge  
ne_map.tsv | sort -u >  
/project/bishalab/msarker/c.auris_new/lncrna_filtering/08_expression_screen/results/expression_  
pass_transcript_ids.txt
```

Count the transcript ids

```
> wc -l  
/project/bishalab/msarker/c.auris_new/lncrna_filtering/08_expression_screen/results/expression_  
pass_transcript_ids.txt
```

Count-614. Here, 613 passing gene locus corresponds to 614 transcript isoforms, meaning one passing locus has two transcript models.

Now extract the 614 transcripts:

```
> gffread  
/project/bishalab/msarker/c.auris_new/lncrna_filtering/06_pfam/results/primary_ux_pfam_pass.g  
tf --ids  
/project/bishalab/msarker/c.auris_new/lncrna_filtering/08_expression_screen/results/expression_  
pass_transcript_ids.txt -T -F -o  
/project/bishalab/msarker/c.auris_new/lncrna_filtering/08_expression_screen/results/expression_  
pass_candidates.gtf
```

Verify the transcript number:

```
> awk -F'\t' '$3=="transcript"{n++;} END{print n+0}'  
/project/bishalab/msarker/c.auris_new/lncrna_filtering/08_expression_screen/results/expression_  
pass_candidates.gtf
```

Verify unique loci:

```
awk -F'\t' '$3=="transcript">{print $9}'  
/project/bishalab/msarker/c.auris_new/lncrna_filtering/08_expression_screen/results/expression_  
pass_candidates.gtf | sed -n 's/.*gene_id "([^"]*)".*/\1/p' | sort -u | wc -l
```

Step 29: Identify which of the 613 loci belong to the 34 coding-gene 3'- risk transcripts.

Converted the risk transcript IDs to gene or locus IDs:

```
> awk 'NR==FNR {risk[$1]=1; next} ($1 in risk){print $2}'  
/project/bishalab/msarker/c.auris_new/lncrna_filtering/07_genomic_context/coding_3prime_risk
```

```
_ids.txt
/project/bishalab/msarker/c.auris_new/lncrna_filtering/06_pfam/results/pfam_pass_transcript_gene_map.tsv | sort -u >
/project/bishalab/msarker/c.auris_new/lncrna_filtering/07_genomic_context/coding_3prime_risk_gene_ids.txt
```

Checked how many unique risk loci they represent:

```
> wc -l
/project/bishalab/msarker/c.auris_new/lncrna_filtering/07_genomic_context/coding_3prime_risk_gene_ids.txt
```

Output: 33

Now intersect them with the 613 expression-passing loci. Here “Intersect” means finding IDs present in both (expression pass & 3’ risk) lists.

```
> awk 'NR==FNR{pass[$1]=1; next} ($1 in pass){print $1}'
/project/bishalab/msarker/c.auris_new/lncrna_filtering/08_expression_screen/results/expression_pass_gene_ids.txt
/project/bishalab/msarker/c.auris_new/lncrna_filtering/07_genomic_context/coding_3prime_risk_gene_ids.txt | sort -u >
/project/bishalab/msarker/c.auris_new/lncrna_filtering/07_genomic_context/risk_expression_pass_gene_ids.txt
```

Count the risk loci requiring examination:

```
wc -l
/project/bishalab/msarker/c.auris_new/lncrna_filtering/07_genomic_context/risk_expression_pass_gene_ids.txt
```

Counts- 32

Summary

Category	Expression pass	Expression fails	Total
3'-risk loci	32	1	33
Non-risk loci	581	43	624
Total	613	44	657

Then we converted the 32 expression-passing risk gene/locus IDs into their corresponding transcript IDs

```
> awk 'NR==FNR{keep[$1]=1; next} ($2 in keep){print $1}'
/project/bishalab/msarker/c.auris_new/lncrna_filtering/07_genomic_context/risk_expression_pass_gene_ids.txt
```

```
s_gene_ids.txt
/project/bishalab/msarker/c.auris_new/lncrna_filtering/06_pfam/results/pfam_pass_transcript_gene_map.tsv | sort -u >
/project/bishalab/msarker/c.auris_new/lncrna_filtering/07_genomic_context/risk_expression_pass_transcript_ids.txt
```

checked the number:

```
> wc -l
/project/bishalab/msarker/c.auris_new/lncrna_filtering/07_genomic_context/risk_expression_pass_transcript_ids.txt
```

Then extracted their GTF:

```
> gffread
/project/bishalab/msarker/c.auris_new/lncrna_filtering/06_pfam/results/primary_ux_pfam_pass.gtf --ids
/project/bishalab/msarker/c.auris_new/lncrna_filtering/07_genomic_context/risk_expression_pass_transcript_ids.txt -T -F -o
/project/bishalab/msarker/c.auris_new/lncrna_filtering/07_genomic_context/risk_expression_pass_candidates.gtf
```

Verify the extracted GTF:

```
> awk -F'\t' '$3=="transcript"{n++} END{print "Transcripts:",n+0}'
/project/bishalab/msarker/c.auris_new/lncrna_filtering/07_genomic_context/risk_expression_pass_candidates.gtf

> awk -F'\t' '$3=="transcript"{print $9}'
/project/bishalab/msarker/c.auris_new/lncrna_filtering/07_genomic_context/risk_expression_pass_candidates.gtf | sed -n 's/.*gene_id "([^"]*)".*/\1/p' | sort -u | wc -l
```

The next step is to connect these candidates with their nearby coding-gene partners before IGV review.

First inspect the existing risk-pair table:

```
> head -n 5
/project/bishalab/msarker/c.auris_new/lncrna_filtering/07_genomic_context/coding_3prime_risk_pairs.tsv | column -t
```

Check the number of columns:

```
> awk 'NF{print "Number of columns:",NF; exit}'
/project/bishalab/msarker/c.auris_new/lncrna_filtering/07_genomic_context/coding_3prime_risk_pairs.tsv
```

Then created a review table for only the 33 expression-passing risk transcripts. First generated an intermediate table and classify each pair as downstream or overlapping the coding-gene 3' boundary:

```
> awk 'BEGIN{OFS="\t"} NR==FNR{keep[$1]=1; next} ($4 in
keep){if($12==""){endpoint=$8; if($2>=endpoint){relation="DOWNSTREAM"; distance=$2-
endpoint}else if($3>endpoint){relation="SPAN_OVERLAPS_3PRIME_END";
distance=0}else{relation="OTHER"; distance=endpoint-$3}}else{endpoint=$9;
if($3<=endpoint){relation="DOWNSTREAM"; distance=endpoint-$3}else
if($2<endpoint){relation="SPAN_OVERLAPS_3PRIME_END";
distance=0}else{relation="OTHER"; distance=$2-endpoint}} print
$4,$1,$2+1,$3,$6,$10,$7,$8+1,$9,$12,relation,distance}'
/project/bishalab/msarker/c.auris_new/lncrna_filtering/07_genomic_context/risk_expression_pas
s_transcript_ids.txt
/project/bishalab/msarker/c.auris_new/lncrna_filtering/07_genomic_context/coding_3prime_risk
_pairs.tsv >
/project/bishalab/msarker/c.auris_new/lncrna_filtering/07_genomic_context/risk_expression_pas
s_pairs.tmp.tsv
```

Added each candidate's MSTRG gene ID and a header:

```
> awk 'BEGIN{OFS="\t"; print
"candidate_transcript_id","candidate_gene_id","candidate_chr","candidate_start","candidate_end
","candidate_strand","coding_gene_id","window_chr","window_start","window_end","coding_s
trand","relationship","distance_bp"} NR==FNR{gene[$1]=$2; next} {print
$1,gene[$1],$2,$3,$4,$5,$6,$7,$8,$9,$10,$11,$12}'
/project/bishalab/msarker/c.auris_new/lncrna_filtering/06_pfam/results/pfam_pass_transcript_ge
ne_map.tsv
/project/bishalab/msarker/c.auris_new/lncrna_filtering/07_genomic_context/risk_expression_pas
s_pairs.tmp.tsv >
/project/bishalab/msarker/c.auris_new/lncrna_filtering/07_genomic_context/risk_expression_pas
s_review.tsv
```

Confirmed the expected transcripts and loci:

```
> awk -F'\t' 'NR>1{print $1}'
/project/bishalab/msarker/c.auris_new/lncrna_filtering/07_genomic_context/risk_expression_pas
s_review.tsv | sort -u | wc -l
```

Summarize the relationships

```
> awk -F'\t' 'NR>1{n[$12]++} END{for(x in n) print x,n[x]}'
/project/bishalab/msarker/c.auris_new/lncrna_filtering/07_genomic_context/risk_expression_pas
s_review.tsv
```

There are still only 33 candidate transcripts, because MSTRG.36939.1 has two coding-gene partners and therefore contributes two pair records.

Now created IGV viewing regions containing the candidate, coding-gene 3' window and an additional 1-kb flank:

```
> awk -F'\t' 'BEGIN{OFS="\t"; print
"IGV_region","candidate_transcript_id","candidate_gene_id","coding_gene_id","relationship","
distance_bp"} NR>1 {start=($4<$9?$4:$9)-1000; if(start<1)start=1;
stop=($5>$10?$5:$10)+1000; region=sprintf("%s:%d-%d",$3,start,stop); print
region,$1,$2,$7,$12,$13}'
/project/bishalab/msarker/c.auris_new/lncrna_filtering/07_genomic_context/risk_expression_pas
s_review.tsv >
/project/bishalab/msarker/c.auris_new/lncrna_filtering/07_genomic_context/igv_review_regions.
tsv
```

Verify:

```
> head -n 10
/project/bishalab/msarker/c.auris_new/lncrna_filtering/07_genomic_context/igv_review_regions.
tsv | column -t
```

Also checked the number of rows:

```
> wc -l
/project/bishalab/msarker/c.auris_new/lncrna_filtering/07_genomic_context/igv_review_regions.
tsv
```

You will get **35** lines: **one** header plus **34** candidate–coding-gene pair records. The file may contain 34 records despite having **only 33 unique transcripts** because one transcript has two coding-gene partners.

Step 30: IGV analysis

prepared a small IGV review package instead of loading the complete GENCODE annotation.

Extract the coding-gene partner IDs

```
> awk -F'\t' 'NR>1 {print $7}'
/project/bishalab/msarker/c.auris_new/lncrna_filtering/07_genomic_context/risk_expression_pas
s_review.tsv | sort -u >
/project/bishalab/msarker/c.auris_new/lncrna_filtering/07_genomic_context/coding_partner_gen
e_ids.txt
```

Check their number:

```
> wc -l
/project/bishalab/msarker/c.auris_new/lncrna_filtering/07_genomic_context/coding_partner_gene_ids.txt
```

Extract their GENCODE annotations

Create exact search patterns:

```
> awk '{print "gene_id \""$1"\""}'
/project/bishalab/msarker/c.auris_new/lncrna_filtering/07_genomic_context/coding_partner_gene_ids.txt >
/project/bishalab/msarker/c.auris_new/lncrna_filtering/07_genomic_context/coding_partner_gene_patterns.txt
```

Extract the annotations:

```
> grep -F -f
/project/bishalab/msarker/c.auris_new/lncrna_filtering/07_genomic_context/coding_partner_gene_patterns.txt /project/bishalab/msarker/c.auris_new/gencode.v48.annotation.gtf >
/project/bishalab/msarker/c.auris_new/lncrna_filtering/07_genomic_context/coding_partner_annotations.gtf
```

Combine coding partners with the 33 risk transcripts

```
> gffread
/project/bishalab/msarker/c.auris_new/lncrna_filtering/07_genomic_context/coding_partner_annotations.gtf
/project/bishalab/msarker/c.auris_new/lncrna_filtering/07_genomic_context/risk_expression_pass_candidates.gtf -T -F -o
/project/bishalab/msarker/c.auris_new/lncrna_filtering/07_genomic_context/igv_risk_annotation.gtf
```

Convert the review regions to BED

```
> awk -F'\t' 'BEGIN{OFS="\t"} NR>1 {split($1,a,":"); split(a[2],b,"-"); print a[1],b[1]-1,b[2]}'
/project/bishalab/msarker/c.auris_new/lncrna_filtering/07_genomic_context/igv_review_regions.tsv | sort -k1,1 -k2,2n | bedtools merge -i - >
/project/bishalab/msarker/c.auris_new/lncrna_filtering/07_genomic_context/igv_review_regions.bed
```

verify

```
> head
/project/bishalab/msarker/c.auris_new/lncrna_filtering/07_genomic_context/igv_review_regions.bed
```

These files will be used next in the IGV desktop application:

- `igv_risk_annotation.gtf`: candidate and nearby coding-gene annotations
- `igv_review_regions.tsv`: searchable regions and candidate identities
- `igv_review_regions.bed`: regions for extracting small review BAM files

The next step is to create reduced BAM files containing reads only from these regions, making IGV review much faster than transferring all 24 complete BAMs.

Create the output and log directories:

```
mkdir -p  
/project/bishalab/msarker/c.auris_new/lncrna_filtering/07_genomic_context/igv_bams/logs
```

Created `igv_subset.sh`

Submit it:

```
dos2unix igv_subset.sh
```

```
sbatch igv_subset.sh
```

After completion, we verified if that 24 reduced BAMs and 24 indexes were generated:

```
> find /project/bishalab/msarker/c.auris_new/lncrna_filtering/07_genomic_context/igv_bams -  
maxdepth 1 -name "*_IGV_regions.bam" | wc -l
```

```
> find /project/bishalab/msarker/c.auris_new/lncrna_filtering/07_genomic_context/igv_bams -  
maxdepth 1 -name "*_IGV_regions.bam.bai" | wc -l
```

Check BAM integrity:

```
> samtools quickcheck -v  
/project/bishalab/msarker/c.auris_new/lncrna_filtering/07_genomic_context/igv_bams/*_IGV_re  
gions.bam
```

No output means all **BAMs** passed. We then transferred the reduced **BAMs**, their **.bai** indexes, `igv_risk_annotation.gtf`, and `igv_review_regions.tsv` to your local computer for IGV review.

The IGV version was **2.19.8**; we choose the Windows package that includes Java to avoid installing Java separately.

In IGV, selected Genomes **Human hg38**.

Loaded all 24 files ending in: `*_IGV_regions.bam`

For the desktop version, we do not need to select the **.bai** files manually; IGV should find them automatically when they are in the same folder.

Opened `igv_review_regions.tsv` in Excel. Then, for each row in `igv_review_regions.tsv`:

- Paste the IGV_region coordinate into IGV's search box.
- **Expand** igv_risk_annotation.gtf. (right click and select expand).
- Compare the **candidate** with the nearby **coding gene**.
- Examine whether the **candidate signal** is visible across multiple BAMs.
- Record **RETAIN**, **EXCLUDE_READTHROUGH**, **EXCLUDE_EXTENSION**, or **AMBIGUOUS**.

RETAIN: separate reproducible signal with a gap.

EXCLUDE_READTHROUGH: continuous signal from the coding gene.

EXCLUDE_EXTENSION: candidate appears to extend the terminal coding exon.

AMBIGUOUS: insufficient evidence to decide.

Final outcome

There are 7 non-retain review rows, but only 6 unique transcripts to discard.

MSTRG.36939.1 appears twice because it was evaluated against two coding genes. The six strict catalogue exclusions are:

- MSTRG.1803.1
- MSTRG.17016.1
- MSTRG.22578.15
- MSTRG.34694.1
- MSTRG.36939.1 (**shares 2 genes on the same strand, chimeric extension**)
- MSTRG.13391.1

Created the exclusion list:

```
> mkdir -p /project/bishalab/msarker/c.auris_new/lncrna_filtering/09_final_catalog
> printf '%s\n' MSTRG.1803.1 MSTRG.17016.1 MSTRG.22578.15 MSTRG.34694.1
MSTRG.36939.1 MSTRG.13391.1 >
/project/bishalab/msarker/c.auris_new/lncrna_filtering/09_final_catalog/igv_excluded_transcript_ids.txt
```

Removed them from the 614 expression-passing transcripts:

```
> grep -F -x -v -f
/project/bishalab/msarker/c.auris_new/lncrna_filtering/09_final_catalog/igv_excluded_transcript_ids.txt
/project/bishalab/msarker/c.auris_new/lncrna_filtering/08_expression_screen/results/expression_pass_transcript_ids.txt >
```

```
/project/bishalab/msarker/c.auris_new/lncrna_filtering/09_final_catalog/final_novel_lncRNA_transcript_ids.txt
```

Verify:

```
> wc -l
/project/bishalab/msarker/c.auris_new/lncrna_filtering/09_final_catalog/igv_excluded_transcript_ids.txt
/project/bishalab/msarker/c.auris_new/lncrna_filtering/09_final_catalog/final_novel_lncRNA_transcript_ids.txt
```

Next, extract the 608 retained transcripts from the Pfam-pass GTF.

```
> gffread
/project/bishalab/msarker/c.auris_new/lncrna_filtering/06_pfam/results/primary_ux_pfam_pass.gtf --ids
/project/bishalab/msarker/c.auris_new/lncrna_filtering/09_final_catalog/final_novel_lncRNA_transcript_ids.txt -T -F -o
/project/bishalab/msarker/c.auris_new/lncrna_filtering/09_final_catalog/final_novel_lncRNA_MSTRG.gtf
```

Verify the transcript count:

```
> awk '$3=="transcript" {n++} END {print n+0}'
/project/bishalab/msarker/c.auris_new/lncrna_filtering/09_final_catalog/final_novel_lncRNA_MSTRG.gtf
```

Verify the unique locus count:

```
> awk -F'\t' '$3=="transcript" {print $9}'
/project/bishalab/msarker/c.auris_new/lncrna_filtering/09_final_catalog/final_novel_lncRNA_MSTRG.gtf | sed -n 's/.*gene_id "([^"]*)".*/\1/p' | sort -u | wc -l
```

Explicitly label the MSTRG records as novel_lncRNA. Create the labeled novel GTF:

```
> awk -F'\t' 'BEGIN{OFS="\t"} /^#/ {print;next} {if($9~/gene_type "/)$9=$9 " gene_type \"novel_lncRNA\""; if($9~/transcript_type "/)$9=$9 " transcript_type \"novel_lncRNA\""; print}'
/project/bishalab/msarker/c.auris_new/lncrna_filtering/09_final_catalog/final_novel_lncRNA_MSTRG.gtf >
/project/bishalab/msarker/c.auris_new/lncrna_filtering/09_final_catalog/final_novel_lncRNA_labeled.gtf
```

Verify the label:

```
> awk -F'\t' '$3=="transcript" && $9~/gene_type "novel_lncRNA"/{n++} END{print n+0}'
/project/bishalab/msarker/c.auris_new/lncrna_filtering/09_final_catalog/final_novel_lncRNA_labeled.gtf
```

Combine full GENCODE v48 with the final labeled novel GTF:

```
> awk 'NR==FNR {print;next} !/^#/'
/project/bishalab/msarker/c.auris_new/gencode.v48.annotation.gtf
/project/bishalab/msarker/c.auris_new/lncrna_filtering/09_final_catalog/final_novel_lncRNA_labeled.gtf >
/project/bishalab/msarker/c.auris_new/lncrna_filtering/09_final_catalog/final_annotation_GENC
ODEv48_plus_novel_lncRNA.gtf
```

Verify the **novel portion** inside the combined annotation:

```
> awk -F'\t' '$3=="transcript" && $9~/gene_type "novel_lncRNA"/{n++; if($1=="chrM")m++}
END{print "Novel transcripts:",n+0; print "Novel chrM transcripts:",m+0}'
/project/bishalab/msarker/c.auris_new/lncrna_filtering/09_final_catalog/final_annotation_GENC
ODEv48_plus_novel_lncRNA.gtf
```

Then ran the final featureCounts using the combined annotation and all 24 stranded BAM files.

First create the output directory:

```
mkdir -p /project/bishalab/msarker/c.auris_new/lncrna_filtering/09_final_catalog/final_counts
```

Create **final_featurecounts.sh**

```
dos2unix final_featurecounts.sh
```

```
sbatch final_featurecounts.sh
```

Here, **-s 1** applies strand-aware counting for your QuantSeq **FWD libraries**

confirm the matrix has 24 sample columns:

```
> awk -F'\t' '!/^#/{print "Total columns:",NF; exit}'
/project/bishalab/msarker/c.auris_new/lncrna_filtering/09_final_catalog/final_counts/final_gene_
counts.txt
```

Step 31: Differential Expression Analysis in R

Files required:

1. gene_counts
2. metadata
3. final_gtf

Created a **gene-annotation table** in bash terminal so R knows which rows are protein-coding, known lncRNA, novel lncRNA, or another biotype.

```
> awk -F'\t' 'BEGIN{OFS="\t"; print "gene_id","gene_name","gene_type","Chr","Strand"} !/^#/\
&& ($3=="gene" || $3=="transcript"){gid=gname=gtype=""; if(match($9,/gene_id "[^"]+"/))
gid=substr($9,RSTART+9,RLENGTH-10); if(match($9,/gene_name "[^"]+"/))
gname=substr($9,RSTART+11,RLENGTH-12); if(match($9,/gene_type "[^"]+"/))
gtype=substr($9,RSTART+11,RLENGTH-12); if(gid!="" && !seen[gid]++){if(gname=="")
gname=gid; if(gtype=="") gtype=(gid~/^MSTRG\./?"novel_lncRNA":"unknown"); print
gid,gname,gtype,$1,$7}}'
/project/bishalab/msarker/c.auris_new/lncrna_filtering/09_final_catalog/final_annotation_GENC
ODEv48_plus_novel_lncRNA.gtf >
/project/bishalab/msarker/c.auris_new/lncrna_filtering/09_final_catalog/final_gene_annotation.ts
v
```

verify

```
> wc -l
/project/bishalab/msarker/c.auris_new/lncrna_filtering/09_final_catalog/final_gene_annotation.ts
v
```

Output: 79294 (one header plus 79,293 genes)

Also verified the novel-lncRNA loci:

```
> awk -F'\t' 'NR>1 && $3=="novel_lncRNA"{n++} END{print n+0}'
/project/bishalab/msarker/c.auris_new/lncrna_filtering/09_final_catalog/final_gene_annotation.ts
v
```

counts: 607

Now see R files analysis_part_1.R and analysis_part_1.RData.

All the downstream analysis of UpSet overlapping, Enrichment, Co-expression, temporal interaction analysis, cis-proximity and co-expression base network was done and plotted in those 2 data files.

Differential expressions were assessed using donor-adjusted quasi-likelihood F-tests in edgeR. P-values were adjusted using the Benjamini-Hochberg method. Genes with $\text{FDR} < 0.05$ were considered statistically differentially expressed, while genes additionally exhibiting an absolute $\log_2$ fold change greater than 1 were designated large-effect DEGs for visualization and candidate prioritization. The class columns occasionally do not equal total DE because some genes are classified as "other".